

Comments

CLEAN CODE-CHAPTER 4

Comments

- ▶ Nothing can be quite so helpful as a well-placed comment. Nothing can clutter up a module more than frivolous dogmatic comments. Nothing can be quite so damaging as an old comment that propagates lies and misinformation.
- ▶ If our programming languages were expressive enough, or if we had the talent to subtly wield those languages to express our intent, we would not need comments very much—perhaps not at all.

Comments Do Not Make Up for Bad Code

Clear and expressive code with few comments is far superior to cluttered and complex code with lots of comments. Rather than spend your time writing the comments that explain the mess you've made, spend it cleaning that mess.

Comments Do Not Make Up for Bad Code

```
// Check to see if the employee is eligible for full benefits  
if ((employee.flags & HOURLY_FLAG) &&  
    (employee.age > 65))
```

Or this?

```
if (employee.isEligibleForFullBenefits())
```

Some comments are necessary or beneficial. However the only truly good comment is the comment you found a way not to write.

Legal Comments

Sometimes our corporate coding standards force us to write certain comments for legal reasons. For example, copyright and authorship statements are necessary and reasonable things to put into a comment at the start of each source file.

- ▶ **Copyright Statements:** These comments typically specify who holds the copyright for the code, protecting intellectual property and ensuring the company or individuals are legally credited for their work.
- ▶ **Authorship Statements:** These comments often include information about who wrote the code, which can be important for tracking contributions, managing software development, and understanding who to approach for future questions or updates related to the code.

Informative Comments

It is sometimes useful to provide basic information with a comment. For example, consider this comment that explains the return value of an abstract method:

```
// Returns an instance of the Responder being tested.  
protected abstract Responder responderInstance();
```

Clarification

- ▶ Sometimes it is just helpful to translate the meaning of some obscure argument or return value into something that's readable
- ▶ In general it is better to find a way to make that argument or return value clear in its own right; but when its part of the standard library, or in code that you cannot alter, then a helpful clarifying comment can be useful

Warning

Sometimes it is useful to warn other programmers about certain consequences.

```
// Don't run unless you
```

```
// have some time to kill.
```

```
public void _testWithReallyBigFile() {  
    writeLinesToFile(100000000);  
    response.setBody(testFile);  
    response.readyToSend(this);  
    String responseString = output.toString();  
    assertSubString("Content-Length: 1000000000", responseString);  
    assertTrue(bytesSent > 1000000000);  
}
```


TODO Comments

It is sometimes reasonable to leave “To do” notes in the form of //TODO comments. In the following case, the TODO comment explains why the function has a degenerate implementation and what that function's future should be

```
//TODO-MdM these are not needed  
// We expect this to go away when we do the checkout model  
protected VersionInfo makeVersion() throws Exception {  
    return null;  
}
```

Amplification

A comment may be used to amplify the importance of something that may otherwise seem inconsequential.

```
String listItemContent = match.group(3).trim();  
// the trim is real important. It removes the starting  
// spaces that could cause the item to be recognized  
// as another list.  
new ListItemWidget(this, listItemContent, this.level + 1);  
return buildList(text.substring(match.end()));
```

Bad Comments

- ▶ Most comments fall into this category. Usually they are crutches or excuses for poor code or justifications for insufficient decisions, amounting to little more than the programmer talking to himself.

Mumbling

Plopping in a comment just because you feel you should or because the process requires it, is a hack. If you decide to write a comment, then spend the time necessary to make sure it is the best comment you can write. Example:

```
public void loadProperties() {  
  
    try {  
        String propertiesPath = propertiesLocation + "/" + PROPERTIES_FILE;  
        FileInputStream propertiesStream = new FileInputStream(propertiesPath);  
        loadedProperties.load(propertiesStream);  
    }  
    catch(IOException e) {  
        // No properties files means all defaults are loaded  
    }  
}
```

Redundant Comments

What purpose does this comment serve? It's certainly not more informative than the code. It does not justify the code, or provide intent or rationale. It is not easier to read than the code. Indeed, it is less precise than the code and entices the reader to accept that lack of precision in lieu of true understanding.

Redundant Comments

```
// Utility method that returns when this.closed is true. Throws an exception
// if the timeout is reached.
public synchronized void waitForClose(final long timeoutMillis) throws Exception {
    if(!closed) {
        wait(timeoutMillis);
        if(!closed)
            throw new Exception("MockResponseSender could not be closed");
    }
}
```